



UFC

UNIVERSIDADE FEDERAL DO CEARÁ

CAMILE ISIDORIO ARAUJO E FRANCISCO WERLEY DA SILVA

RELATÓRIO: TRABALHO 2 - RMI

SISTEMA DE GESTÃO DE RECURSOS HUMANOS

QUIXADÁ

2026

1. INTRODUÇÃO

Este relatório descreve o desenvolvimento do Trabalho 2 da disciplina de Sistemas Distribuídos, que consiste na reimplementação do serviço remoto desenvolvido no Trabalho 1 utilizando o paradigma de Invocação Remota de Método (Remote Method Invocation – RMI), com comunicação cliente-servidor organizada num protocolo de requisição-resposta conforme descrito na seção 5.2 do livro texto.

O serviço implementado é um Sistema de Gestão de Recursos Humanos, que permite o gerenciamento remoto de colaboradores e departamentos de uma empresa. O sistema foi desenvolvido em Java, utilizando JSON como formato de representação externa de dados (marshalling/unmarshalling) e comunicação via sockets TCP, sem uso de frameworks RMI prontos como o java.rmi

2. DESCRIÇÃO DO SERVIÇO REMOTO

O serviço remoto implementado consiste em dois objetos remotos que residem exclusivamente no servidor e são acessíveis pelo cliente via invocação remota de método:

2.1 COLABORADOR SERVICE

Responsável pelo gerenciamento do cadastro de colaboradores da empresa. Expõe os seguintes métodos remotos:

- adicionarColaborador: recebe um objeto Colaborador serializado em JSON (passagem por valor) e o armazena no repositório do servidor.
- buscarColaborador: recebe um id inteiro e retorna o colaborador correspondente serializado em JSON.
- listarColaboradores: retorna a lista completa de colaboradores cadastrados, serializada em JSON.
- removerColaborador: recebe um id e remove o colaborador do repositório, retornando uma mensagem de confirmação.
- calcularFolhaTotal: calcula e retorna o custo total mensal de todos os colaboradores da empresa.

2.2 DEPARTAMENTO SERVICE

Responsável pelo gerenciamento dos departamentos da empresa. Expõe os seguintes métodos remotos:

- criarDepartamento: cria um novo departamento com id, nome e, opcionalmente, um gerente.
- buscarDepartamento: retorna os dados de um departamento pelo id, incluindo seus colaboradores, serializado em JSON.
- adicionarColaboradorAoDepartamento: associa um colaborador existente a um departamento.
- listarDepartamentos: retorna a lista de todos os departamentos com seus membros.

- calcularFolhaDepartamento: calcula o custo total mensal de um departamento específico.

3. MODELAGEM DO SISTEMA

3.1 HIERARQUIA DE CLASSES

O sistema é estruturado a partir da superclasse abstrata Colaborador, que define os atributos e comportamentos comuns a todos os tipos de colaborador. As subclasses especializam esse comportamento:

CLASSE	TIPO	DESCRIÇÃO
COLABORADOR	ABSTRATA	Superclasse com id, nome e salário. Define getTipo() e calcularCustoTotal()
FUNCIONÁRIO	CONCRETA	É-um Colaborador. Possui cargo e data de admissão. Implementa Admissivel
ESTAGIÁRIO	CONCRETA	É-um Colaborador. Possui curso e carga horária semanal
AUTONOMO	CONCRETA	É-um Colaborador. Possui especialidade e CNPJ
EFETIVO	CONCRETA	É-um Funcionario (her. em 2 níveis). Possui bônus anual e anos de empresa
DEPARTAMENTO	CONCRETA	Agrega uma lista de Colaboradores e um Colaborador gerente

3.2 RELAÇÃO ENTRE CLASSES

- Relação é-um (extensão) — 1: Funcionario extends Colaborador, com implementação da interface Admissivel.
- Relação é-um (extensão) — 2: Efetivo extends Funcionario, formando uma hierarquia de dois níveis.
- Relação tem-um (agregação) — 1: Departamento tem uma lista de Colaboradores (membros).
- Relação tem-um (agregação) — 2: Departamento tem um Colaborador gerente.
- Interface Admissivel: Implementada por Funcionario e herdada por Efetivo, define os métodos admitir() e getDataAdmissao(), permitindo polimorfismo.

3.3 ESTRUTURA DE PACOTES

PACOTE	CLASSES	RESPONSABILIDADE
COMMON	Colaborador, Funcionario, Estagiario, Autonomo, Efetivo, Departamento, Admissivel, RemoteObjectRef, JsonSerializer	Entidades compartilhadas entre cliente e servidor, mais a serialização JSON
PROTOCOL	Message, RequestReplyProtocol	Protocolo de requisição-resposta com os métodos doOperation, getRequest e sendReply
SERVER	RHServer, ColaboradorService, DepartamentoService	Servidor TCP, dispatcher e objetos de serviço remotos
CLIENT	RHClient	Cliente com menu interativo em modo texto

4. PROTOCOLO DE REQUISIÇÃO-RESPOSTA

4.1 ESTRUTURA DA MENSAGEM

As mensagens trocadas entre cliente e servidor seguem a estrutura definida na seção 5.2 do livro texto e são implementadas pela classe Message, com os seguintes campos:

CAMPO	TIPO	DESCRIÇÃO
messageType	int	0 = REQUEST, 1 = REPLY
requestId	int	Identificador único da requisição (gerado atomicamente)
objectReference	String	Nome do objeto remoto: ColaboradorService ou DepartamentoService
methodId	String	Nome do método a ser invocado
arguments	String(JSON)	Argumentos ou resultado serializado em JSON

A mensagem é serializada em JSON e transmitida com um cabeçalho de 4 bytes contendo o tamanho do payload, garantindo leitura segura e completa dos dados no socket TCP.

4.2 MÉTODOS DO PROTOCOLO

A classe `RequestReplyProtocol` implementa exatamente os três métodos exigidos pelo enunciado:

- `doOperation(RemoteObjectRef, String methodId, byte[] arguments)`: Executado no cliente. Abre um socket TCP, envia a mensagem de requisição e aguarda a resposta bloqueadamente.
- `getRequest(DataInputStream in)`: Executado no servidor. Lê os bytes de uma requisição de um cliente através da porta servidora.
- `sendReply(byte[] reply, int requestId, DataOutputStream out)`: Executado no servidor. Envia a mensagem de resposta para o cliente que originou a requisição.

4.3 FLUXO DE COMUNICAÇÃO

O fluxo completo de uma invocação remota ocorre nas seguintes etapas:

- O cliente monta os argumentos em JSON e chama `doOperation()`, passando o `RemoteObjectRef` do serviço desejado.
- O `RequestReplyProtocol` empacota os dados numa `Message` do tipo `REQUEST` e abre uma conexão TCP com o servidor.
- O `RHServer` recebe a conexão, chama `getRequest()` e reconstrói a `Message` a partir dos bytes recebidos.
- O dispatcher analisa o campo `objectReference` e encaminha para `ColaboradorService` ou `DepartamentoService`.
- O serviço executa o método solicitado e retorna o resultado serializado em JSON.
- O `RHServer` chama `sendReply()`, que empacota o resultado numa `Message` do tipo `REPLY` e a envia de volta.
- O cliente recebe os bytes, desserializa o JSON e exibe o resultado no terminal.

5. PASSAGEM DE PARÂMETROS

5.1 PASSAGEM POR REFERÊNCIA

Os objetos de serviço `ColaboradorService` e `DepartamentoService` existem exclusivamente na memória do servidor. O cliente nunca possui uma cópia local desses objetos — ele apenas os referencia através de um `RemoteObjectRef`, que contém host, porta e nome do objeto. Toda operação é executada no servidor, caracterizando a passagem por referência.

5.2 PASSAGEM POR VALOR

As entidades de domínio (`Colaborador` e suas subclasses, `Departamento`) são transmitidas entre cliente e servidor serializadas em JSON pela classe `JsonSerializer`.

A cada chamada remota, uma cópia do objeto é criada e enviada pela rede, caracterizando a passagem por valor. A representação externa de dados em JSON garante interoperabilidade e independência da representação interna Java.

6. REPRESENTAÇÃO EXTERNA DOS DADOS

A representação externa de dados é implementada pela classe `JsonSerializer`, utilizando a biblioteca `org.json`. Para cada tipo de entidade, existem métodos de marshalling (objeto $\rightarrow$ JSON) e unmarshalling (JSON $\rightarrow$ objeto):

- `colaboradorParaJson` / `jsonParaColaborador`: Serializa qualquer subclasse de `Colaborador`, incluindo o campo `tipo` para reconstrução polimórfica.
- `departamentoParaJson` / `jsonParaDepartamento`: Serializa o departamento com seu gerente e lista de colaboradores aninhados.
- `refParaJson` / `jsonParaRef`: Serializa o `RemoteObjectRef` para transporte na mensagem.

A classe `Message` também possui seus próprios métodos `toBytes()` e `fromBytes()`, que serializam a própria mensagem do protocolo em JSON com framing de 4 bytes para transporte seguro via socket TCP.

7. CONCLUSÃO

O trabalho proporcionou a compreensão prática do paradigma de Invocação Remota de Método, evidenciando como o protocolo de requisição-resposta abstrai a comunicação em rede do ponto de vista da aplicação. A separação clara entre passagem por valor e por referência demonstra os fundamentos da distribuição de objetos em sistemas distribuídos.

A utilização de JSON como formato de representação externa de dados mostrou-se adequada pela legibilidade e facilidade de depuração durante o desenvolvimento, além de satisfazer o requisito de independência da representação interna Java. A estrutura do projeto em pacotes bem definidos (`common`, `protocol`, `server`, `client`) facilitou a organização e a manutenção do código, tornando explícitas as responsabilidades de cada camada da arquitetura.